

TREASURE: The Visa Payment Foundation Model for High-Volume Transaction Understanding

Chin-Chia Michael Yeh*
Uday Singh Saini
Xin Dai
Xiran Fan
Shubham Jain
Visa Research
Foster City, CA, USA

Yujie Fan
Jiarui Sun
Junpeng Wang
Menghai Pan
Yingtong Dou
Visa Research
Foster City, CA, USA

Yuzhong Chen
Vineeth Rakesh
Liang Wang
Yan Zheng
Mahashweta Das
Visa Research
Foster City, CA, USA

Abstract

Payment networks form the backbone of modern commerce, generating high volumes of transaction records from daily activities. Properly modeling this data can enable applications such as abnormal behavior detection and consumer-level insights for hyper-personalized experiences, ultimately improving people’s lives. In this paper, we present TREASURE, TRansformer Engine As Scalable Universal transaction Representation Encoder, a multi-purpose transformer-based foundation model specifically designed for transaction data. The model simultaneously captures both consumer behavior and payment network signals (such as response codes and system flags), providing comprehensive information necessary for applications like accurate recommendation systems and abnormal behavior detection. Verified with industry-grade datasets, TREASURE features three key capabilities: 1) an input module with dedicated sub-modules for static and dynamic attributes, enabling more efficient training and inference; 2) an efficient and effective training paradigm for predicting high-cardinality categorical attributes; and 3) demonstrated effectiveness as both a standalone model that increases abnormal behavior detection performance by 111% over production systems and an embedding provider that enhances recommendation models by 104%. We present key insights from extensive ablation studies, benchmarks against production models, and case studies, highlighting valuable knowledge gained from developing TREASURE.

CCS Concepts

• **Computing methodologies** → **Neural networks; Temporal reasoning**; • **Information systems** → **Data mining**.

Keywords

Payment Network, Foundation Model, Sequential Tabular Data

1 Introduction

Visa operates as a global payment network company processing over 300B transactions or 15T dollars annually between more than 4B credentials and 150M+ merchants across 200+ countries [28]. Visa deploys 100+ machine learning models that leverage payment network data to ensure transaction integrity and provide valuable services to all parties involved in these transactions [29]. These

machine learning models generally achieve their goals by modeling both cardholder behavior and payment network signals for each transaction. For example, the Stand-in-Processing (STIP) service determines authorization decisions when issuing banks are unavailable to do so [27]. The machine learning model utilized for STIP service needs to distinguish between normal and abnormal cardholder behaviors and predict what signals (i.e., approved or declined) the payment network would send to merchants.

In this paper, we explore the design of a unified foundation model that addresses multiple transaction-related machine learning tasks across multiple sectors. We call our proposed model TREASURE, short for TRansformer Engine As Scalable Universal transaction Representation Encoder. The TREASURE model effectively captures both cardholder behaviors and various payment network signals at the scale of Visa’s data. To achieve this goal, TREASURE incorporates three key capabilities: 1) an input module with dedicated sub-modules for static and dynamic attributes, enabling more efficient training and inference; 2) an efficient and effective training paradigm for predicting high-cardinality categorical attributes; and 3) demonstrated effectiveness both as a standalone system for transaction-based tasks and as an embedding provider for downstream applications.

The first capability addresses the inherent characteristics of transaction data. As shown in Fig. 1, raw transaction data is logged as individual entries whenever transactions occur, with all attributes potentially changing between transactions.

Time	Card #	Issue Bank	...	Card Product	Merchant	...	Amount
1652141652	YYYY	Bank Y	...	Infinite	Merchant F	...	\$214.74
1652141656	XXXX	Bank X	...	Signature	Merchant C	...	\$13.24
1652141728	YYYY	Bank Y	...	Infinite	Merchant D	...	\$25.63
1652141749	YYYY	Bank Y	...	Infinite	Merchant C	...	\$63.28
1652141750	XXXX	Bank X	...	Signature	Merchant A	...	\$101.57
1652141923	XXXX	Bank X	...	Signature	Merchant C	...	\$15.12
1652142151	XXXX	Bank X	...	Signature	Merchant B	...	\$1.26
...	...	...	...	...	...	...	...

Figure 1: Example of raw transaction data showing interleaved transactions from different cards.

However, since our goal is to model cardholder behavior, it is more meaningful to group transactions from the same card together and order them chronologically. After grouping and sorting, as illustrated in Fig. 2, we observe that certain attributes remain constant

*miyeh@visa.com

throughout a card’s transaction history, while others vary with each transaction. We refer to the unchanging card-associated attributes as *static* attributes, and the changing ones as *dynamic* attributes. As demonstrated in [37], modeling different types of attributes using specialized sub-modules proves both more effective and efficient. Therefore, this first capability is crucial in our development of a foundation model for transaction data.

Time	Card #	Issue Bank	...	Card Product	Merchant	...	Amount
1652141656	XXXX	Bank X	...	Signature	Merchant C	...	\$13.24
1652141750	XXXX	Bank X	...	Signature	Merchant A	...	\$101.57
1652141923	XXXX	Bank X	...	Signature	Merchant C	...	\$15.12
1652142151	XXXX	Bank X	...	Signature	Merchant B	...	\$1.26
1652141652	YYYY	Bank Y	...	Infinite	Merchant F	...	\$214.74
1652141728	YYYY	Bank Y	...	Infinite	Merchant D	...	\$25.63
1652141749	YYYY	Bank Y	...	Infinite	Merchant C	...	\$63.28
...	...	...	...	...	...	...	...

Static Attributes
Dynamic Attributes

Figure 2: Grouped transactions from the same card, demonstrating that attributes can be either static or dynamic when considering the shopping behavior of each cardholder.

The second capability addresses the challenges in training the foundation model. As the model learns cardholder behavior through next-transaction prediction tasks, it must predict all attributes associated with a transaction. This includes categorical attributes with extremely high cardinality. For example, predicting the next merchant using cross-entropy loss would require computing over 150M+ logits for each prediction. Compared to large language models with vocabulary sizes typically under one million [6, 11, 13], our problem is significantly more computationally expensive. Consequently, we cannot simply apply standard cross-entropy loss and must develop a more efficient and effective training paradigm for predicting high-cardinality categorical attributes. In our paper, we provide an efficient solution for approximating cross-entropy loss in the design of TREASURE.

The third capability demonstrates TREASURE’s effectiveness in real-world applications, validating its practical utility. We have tested TREASURE in two distinct scenarios: 1) as a standalone model and 2) as an embedding service provider. When used as a standalone model, we evaluate its abnormal behavior detection capabilities against production models currently deployed in the field, measuring improvements in detection accuracy. Our experiments show that TREASURE can identify suspicious patterns that traditional models might miss, particularly increasing performance by 111%. When used as an embedding provider, we demonstrate how embeddings generated from transaction history can enhance recommendation system performance by 90% through capturing nuanced financial behavior patterns. These embeddings encapsulate temporal spending habits, merchant category preferences, and transaction value distributions, providing a rich foundation for downstream applications without requiring access to the raw transaction data. This dual-purpose capability makes TREASURE particularly valuable as it can serve both specialized security functions and broader

consumer experience applications with the same underlying model architecture.

Our key contributions are as follows:

- Identification of key capabilities required for a transaction foundation model and development of corresponding solutions.
- Analysis of various design choices for both model and dataset construction, including the scaling law for transaction data.
- Demonstration of TREASURE’s superior performance both as a standalone model and as an embedding service.

2 Related Work

Foundation models, defined as models trained at scale on vast datasets, have become a cornerstone of modern AI research [2]. While large language models (LLMs) like GPT are prominent examples [3], the foundation model paradigm has been successfully adapted to other domains, including computer vision [1, 19], time series [5, 20, 33], and tabular data [24, 32]. Our work, TREASURE, builds on these advancements, situating itself at the intersection of foundation models for tabular and sequential transaction data [23].

General-purpose tabular foundation models, such as those in [24, 32], are designed to learn relationships between columns within a static table. However, they are not inherently equipped to model the critical sequential dependencies between rows that characterize transaction data, as illustrated in Fig. 2. The temporal patterns in transaction sequences reflect consumer behavior, which is vital for key applications in the payment industry. Consequently, while these models offer broad applicability, their lack of a sequential focus makes them suboptimal for our specific use case. We do, however, concur with the analysis in [24] regarding the limitations of directly applying LLMs to tabular data, namely their inefficiency in modeling continuous variables and their significant computational expense.

The work most closely related to ours is the transaction foundation model proposed in [23], which also treats transactions as a sequence of events. Despite this similarity, a key distinction sets our work apart: their model does not incorporate signals from the payment network itself. This design choice means their model cannot leverage direct indicators from the payment network, such as issuer decline codes, which provide powerful, real-time evidence that significantly enhances tasks like abnormal behavior detection. Furthermore, their approach relies on Gated Recurrent Units (GRUs) [4], an architecture that our experiments in Section 4.3 demonstrate to be inferior to the Transformer [26] for modeling transaction sequences. To the best of our knowledge, TREASURE is the first Transformer-based foundation model designed to holistically model payment transactions by simultaneously capturing both consumer behavior and payment network signals, validated at an industrial scale.

3 Methodology

This section details the proposed TREASURE model. We begin by outlining the input and output data structures in Section 3.1. Subsequently, we present the overall model architecture in Section 3.2. The core challenge in developing a foundation model for transaction data is the effective modeling of diverse attributes; thus, we dedicate specific subsections to the input module (Section 3.3) and the

output module (Section 3.4). Finally, we describe the optimization objectives in Section 3.5.

3.1 Input and Output

Each data sample comprises an input sequence and two corresponding output sequences, all derived from the transactions associated with a single card. For a card with t transactions, the input sequence consists of a static attribute vector (X_s) and t dynamic attribute vectors ($X_{d,1}, \dots, X_{d,t}$), where each $X_{d,i}$ corresponds to the i -th transaction. Raw attributes are classified as either static or dynamic. Each attribute vector (static or dynamic) contains both numerical and categorical attributes. Numerical attributes are stored as floating-point numbers representing their raw values. Categorical attributes are stored as integers, representing category indices derived from predefined mappings.

For instance, if a transaction amount is \$3.14, the value 3.14 is stored as a floating-point number in the corresponding $X_{d,i}$. If the same transaction includes the numeric-3 country code 840 (United States of America, per ISO 3166-1 numeric [31]), an integer value, such as 58, obtained from a mapping table (e.g., $\{\dots, '834' : 57, '840' : 58, '850' : 59, \dots\}$), is stored in $X_{d,i}$. Given 249 unique numeric-3 country codes, this specific mapping table would map each code to an integer in the range $[0, 248]$, where the range size equals the attribute's cardinality.

For each of the t transactions, TREASURE generates two sets of outputs. The first output set predicts the attributes of the *next* transaction (i.e., transaction $i + 1$). The second output set predicts payment network signals associated with the *current* transaction (i.e., transaction i). These network signals are used exclusively as outputs, as they represent outcomes of a transaction (e.g., a response code indicating approval or decline) that are only known *after* it has been processed. Therefore, given the input at time step i , the model performs two tasks: the *current* prediction head infers the signals for the transaction at time step i , while the *next* prediction head forecasts the attributes for the transaction at time step $i + 1$.

3.2 Overall Architecture

With the input and output structures outlined, we now detail the overall architecture of TREASURE, illustrated in Fig. 3. The model processes two input types—static and dynamic—using two distinct input modules of identical design. The static input module processes the single static vector X_s . The dynamic input module processes each dynamic vector $X_{d,i}$ in the sequence, with its weights shared across all time steps (transactions). The design of these input modules is further detailed in Section 3.3.

After processing by their respective input modules, the resulting intermediate representations are fed into a Transformer decoder block. This block employs causal masked self-attention to capture temporal dependencies within the transaction sequence. To ensure that dynamic transaction vectors can attend to the static information, the static vector's representation is positioned first in the sequence supplied to the Transformer, facilitated by the causal mask. The Transformer decoder block's design is depicted in the inset of Fig. 3. We omit explicit positional encoding, as a decoder-only

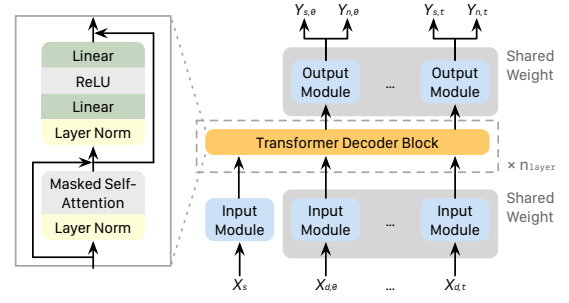


Figure 3: The overall model architecture of TREASURE.

Transformer architecture inherently captures the relative ordering of transactions through its autoregressive nature and causal masking [12, 14].

For each dynamic vector's representation processed by the Transformer, an output module generates the final predictions. Similar to the dynamic input module, the output module's weights are shared across time steps. This output module contains two prediction heads, each dedicated to one of the two output types (the next transaction attributes $Y_{n,i}$ and the current transaction signals $Y_{s,i}$, where $i \in \{0, \dots, t\}$). The detailed design of the output module is presented in Section 3.4.

3.3 Input Module

The detailed architecture of the input module is illustrated in Fig. 4. Its primary function is to model interdependencies among different input attributes within a single transaction vector (either static or one dynamic vector).

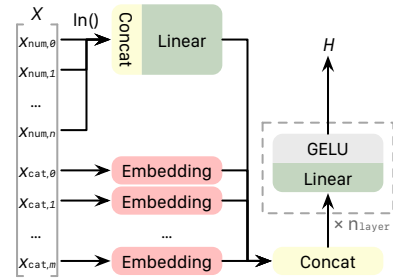


Figure 4: The detailed input module of TREASURE. H represents the input to the Transformer decoder block.

Numerical and categorical attributes are processed differently. Numerical attributes are first transformed to a logarithmic scale, as all numerical features in our dataset (e.g., transaction amounts, time differences between transactions) exhibit long-tail distributions. These log-scaled numerical attributes are then concatenated into a single numerical input vector, which is subsequently processed by a linear layer.

For categorical attributes, each raw value is initially converted to a category index, as described in Section 3.1. This index is directly

used to retrieve a corresponding embedding from an attribute-specific embedding table.¹ We have implemented mechanisms to handle new category values for categorical attributes that may appear during inference but were not present during training. Such OOV (out-of-vocabulary) values can arise from new merchants, emerging business categories, or data quality variations. These mechanisms ensure that an embedding index is consistently available for every categorical attribute value. However, the specific implementation details are omitted to protect sensitive information about the raw category values.

Finally, the representation vector from the numerical linear layer and the embedding vectors for all categorical attributes are concatenated. This combined vector is then processed through additional linear and activation layers to produce a unified representation that captures information across all input attributes.

3.4 Output Module

The detailed structure of the output module is depicted in Fig. 5. This module generates predictions corresponding to each transaction representation output by the Transformer block. It comprises two sub-modules, one for predicting *next* transaction attributes and one for predicting *current* transaction network signals. These sub-modules share an identical architecture, as both must predict numerical and categorical values. They differ only in the specific attributes they target and the temporal origin of their ground truth data.

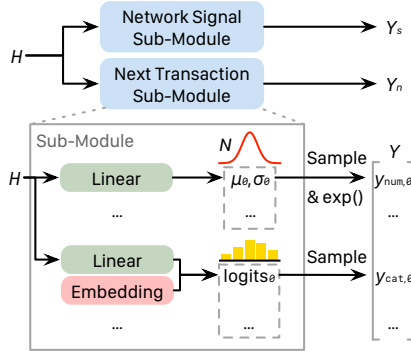


Figure 5: The detailed output module of TREASURE. Two identical sub-modules predict current signals and next transaction attributes using the output of Transformer decoder block, H , as their input.

The input to the output module is the hidden representation H from the Transformer block for a given time step. For numerical attributes, the module predicts a full probability distribution rather than a single point estimate, allowing it to capture the inherent uncertainty in its predictions. This is achieved by estimating the mean (μ) and standard deviation (σ) of the target value in logarithmic scale. This approach effectively models numerical attributes using log-normal distributions—a choice motivated by their long-tail nature in our data [22]. A final predicted value can be obtained

¹In subsequent work, we demonstrated that initializing TREASURE’s categorical embeddings with LLM-based sentence embeddings can further improve model performance [7].

by sampling from this estimated distribution $N(\mu, \sigma)$ and then exponentiating the result.

For categorical attributes, the representation H is first transformed by an attribute-specific linear layer. Thus, if predicting 10 distinct categorical attributes, 10 such linear layers are employed. The output of this linear layer is then used to compute logits for each possible category of that attribute via a vector-matrix multiplication with the attribute’s corresponding embedding table. For instance, to predict one of 249 numeric-3 country codes, the logits are computed by multiplying the output of the country-code-specific linear layer with the country code embedding matrix. A predicted categorical value can be obtained by sampling based on these computed logits.

3.5 Optimization Objective

The learning objective of TREASURE is to accurately predict numerical and categorical attributes for both next-transaction attributes and current-transaction network signals. Consequently, the overall loss function is an aggregation of individual loss terms, each corresponding to a specific predicted attribute. We first define the loss calculation for different attribute types before presenting the aggregate loss.

Attributes are categorized into three types for loss computation: 1) numerical attributes, 2) low-cardinality categorical attributes, and 3) high-cardinality categorical attributes. The distinction between low and high cardinality is determined by a predefined threshold, which is set to 1,024 in this work. In our dataset, categorical attributes range in cardinality from 2 to over 100M. For high-cardinality attributes prone to long-tail distributions and out-of-vocabulary items, we employ a principled aggregation strategy where infrequent or new entities are mapped to shared aggregated identifiers, controlling vocabulary size while maintaining attribute utility. For numerical attributes, we employ the negative log-likelihood of a normal distribution, where the target variable is in logarithmic scale:

$$\mathcal{L}_{\text{num}}(\mu, \sigma, y) = \frac{(y - \mu)^2}{2\sigma^2} + \log(\sigma) + \frac{\log(2\pi)}{2} \quad (1)$$

Here, μ (mean) and σ (standard deviation) are model outputs, and y is the ground truth value, all in logarithmic scale.

For low-cardinality categorical attributes, we use the standard cross-entropy loss:

$$\mathcal{L}_{\text{cat}}(Z, y) = -\log \frac{e^{Z[y]}}{\sum_{i=1}^C e^{Z[i]}} \quad (2)$$

where $Z \in \mathbb{R}^C$ is the vector of logits for C categories, and y is the index of the ground truth category.

For high-cardinality categorical attributes, direct computation of Eq. (2) is infeasible due to the excessive GPU memory required to compute logits for all categories for every attribute, time step, and sample. Instead, we use the InfoNCE loss [17], as defined in Eq. (3). This approach requires computing the logit for only the positive (ground truth) category and a subset of negative categories:

$$\mathcal{L}_{\text{hcat}}(H_a, E, y, I) = -\log \frac{e^{H_a \cdot E[y,:]} }{\sum_{i \in I} e^{H_a \cdot E[i,:]}} \quad (3)$$

where H_a is the output of the attribute-specific linear layer in the output module, E is the embedding matrix for the categorical attribute, y is the positive category’s index, and I is a set containing the index y and indices of sampled negative categories.

Sharing negative indices across all time steps and samples within a batch significantly reduces memory consumption. The pseudocode in Algorithm 1 provides a concrete illustration of this high-cardinality loss computation and its sharing mechanism.

Algorithm 1 The function computes loss for high-cardinality categorical attributes.

```

1  def high_cardinality_loss(hidden, embedding, label_positive, n_negative):
2      """
3      Input:
4          hidden: (batch_size, sequence_length, hidden_dimension)
5          embedding: (n_category, hidden_dimension)
6          label_positive: (batch_size, sequence_length)
7          n_negative: int
8      Output:
9          loss: (batch_size, sequence_length)
10     """
11     hidden_positive = embedding[label_positive, :]
12     label_negative = torch.randint(
13         embedding.shape[0], (n_negative, ),
14         dtype=torch.long, device=label_positive.device)
15     hidden_negative = embedding[label_negative, :]
16     dot_positive = torch.einsum(
17         'ijk,ljk->ilj', hidden, hidden_positive)
18     dot_negative = torch.einsum(
19         'ijk,lk->ilj', hidden, hidden_negative)
20     dot_all = torch.cat([dot_positive, dot_negative], dim=1)
21     indices = torch.arange(hidden.shape[0])
22     loss_numerator = dot_positive[indices, indices]
23     loss_denominator = torch.logsumexp(dot_all, dim=1)
24     return loss_numerator - loss_denominator

```

To validate the efficacy of our shared negative sampling strategy, we compared its memory footprint against a baseline using independent negative samples for each loss computation. Memory usage for both forward and backward passes was measured across varying numbers of negative samples, as shown in Fig. 6. The experiment was conducted with PyTorch 2.6. The shared sampling method maintained forward pass memory below 3GB and backward pass memory below 6GB. In contrast, the independent sampling method encountered out-of-memory errors when the number of negative samples exceeded 64. The memory usage for configurations resulting in out-of-memory errors is extrapolated linearly from the last successful runs.

With individual attribute loss terms defined, we now describe their aggregation into the overall loss. The abnormal behavior flag is considered the most critical attribute, indicating transaction normality, i.e., whether a transaction should be considered part of a user’s normal behavior. Its corresponding loss, $\mathcal{L}_{\text{abnormal}}$, serves as a reference to scale other attribute losses. The overall loss $\mathcal{L}$ is formulated as:

$$\mathcal{L} = \mathcal{L}_{\text{abnormal}} + \frac{1}{|\mathbf{L}|} \sum_{\mathcal{L}_i \in \mathbf{L}} \min \left(\mathcal{L}_i, \frac{\mathcal{L}_i \hat{\mathcal{L}}_{\text{abnormal}}}{\hat{\mathcal{L}}_i} \right) \quad (4)$$

where $\mathbf{L}$ is the set of all non-abnormal behavior attribute losses. Loss terms with a hat (e.g., $\hat{\mathcal{L}}_{\text{abnormal}}$, $\hat{\mathcal{L}}_i$) denote detached values, meaning gradients are not computed through them when they are used in this scaling mechanism.

The first term is the direct loss for the abnormal behavior flag. The second term aggregates the other losses, dynamically adjusting their contributions. Specifically, for each auxiliary loss $\mathcal{L}_i$, its effective value in the sum is $\mathcal{L}_i$ if $\hat{\mathcal{L}}_i$ is smaller than or equal to $\hat{\mathcal{L}}_{\text{abnormal}}$ (i.e., task i is performing better or similarly). If task i is performing worse (i.e., $\hat{\mathcal{L}}_i > \hat{\mathcal{L}}_{\text{abnormal}}$), its current loss $\mathcal{L}_i$ is scaled down by the ratio $\hat{\mathcal{L}}_{\text{abnormal}}/\hat{\mathcal{L}}_i$. This dynamic loss scaling

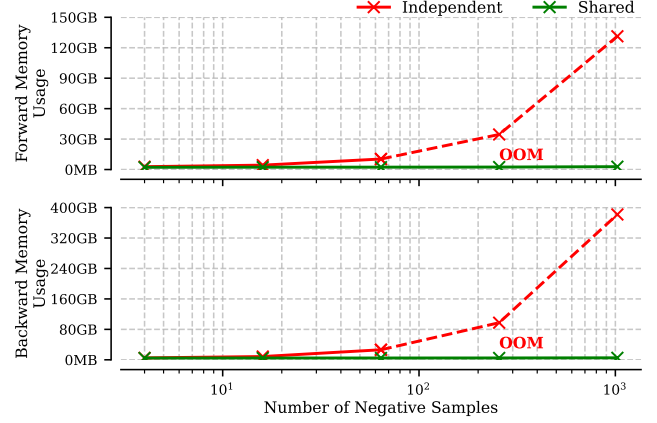


Figure 6: Efficiency improvement through shared negative sampling in TREASURE. By sharing negative samples across both samples and time steps within a batch, we dramatically reduce the memory usage during training. Dashed lines indicate linear extrapolations from the solid lines.

mechanism ensures that the primary abnormal behavior detection task dominates the overall gradient direction while the auxiliary tasks continue to contribute meaningfully to the learning process. By prioritizing the abnormal behavior flag, the model focuses on the most critical attribute while still leveraging auxiliary tasks to improve generalization and representation learning.

4 Experiment

In this section, we first present our experiment setup in Section 4.1. Next, in Section 4.2, we validate the benefits of a multipurpose foundation model compared to single-purpose models and the efficacy of our adopted loss aggregation strategy. We then compare the Transformer backbone used in TREASURE with RNN-based backbones commonly employed in transaction models in Section 4.3. Following this, in a dedicated subsection, we evaluate the effectiveness of different negative sampling strategies. Subsequently, we demonstrate the capability of TREASURE to serve embeddings for specialized applications through a case study in Section 4.5. We also present visualizations of the learned embeddings in Section 4.6. Finally, in Section 4.7, we demonstrate that the performance of TREASURE improves with scale, both in terms of training data volume and model size.

4.1 Dataset and Experiment Setup

We sampled approximately six billion transactions from 30 million distinct cardholders, recorded between September 1, 2020, and November 30, 2022. This dataset spans a total of 26 months. Transactions from the initial 24 months constitute the training data, the 25th month serves as the validation data, and the final month is used as the test data. In the sampled dataset, each transaction comprises five static attributes and sixteen dynamic attributes. During training, TREASURE predicts all sixteen dynamic attributes of the subsequent transaction and two network signals associated with the current transaction. The specific names of these attributes cannot

be disclosed due to the sensitive nature of the project. TREASURE uses a 3-layer Transformer decoder with 4 attention heads and a hidden dimension of 256. The input module also uses 3 layers. The model is trained for 20 epochs using the AdamW optimizer with a learning rate of 10^{-4} , default beta parameters, and a batch size of 256. The best checkpoint is selected based on validation performance.

We assess the model’s performance in predicting the next transaction using precision at one (Prec@1) for selected categorical attributes (i.e., merchant, its country, city, and category) and symmetric mean absolute percentage error (sMAPE) for a selected numerical attribute (i.e., transaction amount). While we report results for selected attributes with high business value and interpretability, our complete evaluations across all attributes show consistent trends. Additionally, we measure the model’s performance in abnormal behavior detection using an in-house performance metric. Detailed performance figures cannot be disclosed due to the sensitive nature of this task. To present these results, we compute the performance ratio between the evaluated model and the currently deployed system. For instance, a ratio of 1.5 signifies that the evaluated model is 50% better than the currently deployed system in abnormal behavior detection. We term this metric Relative Improvement (RI).

For deployment considerations, we cap the sequence length at 512 transactions, which covers over two years of transaction history for most cardholders and provides a deterministic computational budget. Under this configuration, the model’s inference latency remains operationally feasible for real-time transaction processing. The deployment of machine learning models in critical financial infrastructure follows a rigorous multi-stage process involving extensive offline evaluation, parallel validation phases, and regulatory review. The comprehensive offline evaluation results presented in this paper represent a significant milestone in this process, and we are actively working toward full production deployment.

4.2 Optimization Objective

In the first set of experiments, we pursued two primary objectives: 1) to validate the utility of a multipurpose foundation model for transaction data over specialized single-purpose models, and 2) to confirm the necessity of the adopted loss aggregation strategy, i.e., Eq. (4). To achieve the first objective, we trained two single-purpose baseline models. The first baseline focuses on abnormal behavior detection, while the second concentrates on predicting the most likely next merchant. A model’s ability to predict the next merchant serves as a proxy for its performance in recommendation applications. To achieve the second objective, we trained two additional models employing alternative loss aggregation strategies: 1) simple summation of losses, and 2) weighting losses for equal contribution. The experiment results are presented in Table 1.

Comparing the performance of TREASURE with the two single-purpose models, we observe that our method simultaneously outperforms both. This indicates that the proposed foundation model not only conserves resources by enabling a single multi-purpose model to replace multiple single-purpose ones but also delivers superior prediction quality. These results demonstrate the advantages of TREASURE compared to task-specific models.

Table 1: Performance comparisons across different optimization objectives. Arrows indicate improvement direction. Bold and underlined values represent best and second-best results, respectively.

Attribute	Measure	Single Purpose		Multiple Purposes	
		Merchant	Abnormal	Simple	Equal
Abnormal Amount	RI ($\uparrow$)	-	<u>1.9606</u>	1.8768	0.1768
	sMAPE ($\downarrow$)	-	-	0.5850	0.5893
Merchant	Prec@1 ($\uparrow$)	0.1248	-	<u>0.1306</u>	0.1235
Country		-	-	0.5587	0.5132
City		-	-	0.1840	0.2050
Category		-	-	<u>0.4291</u>	0.3672
					0.4335

When contrasting the loss aggregation strategy used in TREASURE with the simple aggregation strategy, our adopted approach yields superior performance across all evaluated aspects. This highlights the necessity of rebalancing the contributions of different loss terms. Compared to the equal contribution strategy, the model trained with this alternative strategy shows improved performance in predicting the next merchant’s city. However, its performance on other attributes is inferior, even to that of the simple aggregation strategy. Notably, the abnormal behavior detection capability, a critical application in payment systems, is significantly degraded by the equal contribution strategy. One plausible explanation is that pivoting each loss term relative to the abnormal detection loss enhances the model’s sensitivity to anomalies, thereby improving its ability to capture sequential patterns in transactions.

In conclusion, the multipurpose foundation model TREASURE is more efficient and effective than multiple single-purpose models. Furthermore, the adopted loss aggregation strategy is substantially more effective than the alternative baseline strategies. Overall, TREASURE demonstrates a significant RI, outperforming the currently deployed system by 111%.

4.3 Temporal Modeling Architecture

While TREASURE utilizes the Transformer architecture [26], existing foundation models for transaction data often employ GRUs [4, 23]. In this set of experiments, we compare variants of TREASURE that utilize different architectures for capturing temporal dependencies to validate the choice of Transformer modules in our method. Specifically, we compared our Transformer-based solution with RNN-based alternatives (i.e., GRU [4] and LSTM [9]). The experiment results are detailed in Table 2.

Table 2: Performance comparisons across different temporal modeling architectures. Arrows indicate improvement direction. Bold and underlined values represent best and second-best results, respectively.

	Abnormal RI ($\uparrow$)	Amount sMAPE ($\downarrow$)	Merchant	Country	City Prec@1 ($\uparrow$)	Category
TREASURE + LSTM	1.4427	0.5850	0.1103	0.5336	0.1546	0.4003
TREASURE + GRU	1.3979	0.6013	0.1172	0.5291	0.1605	0.3945
TREASURE	2.1171	0.5786	0.1421	0.5634	0.1892	0.4335

Both RNN-based solutions yield similar performance across all evaluation measures. However, our Transformer-based solution

consistently outperforms these RNN-based variants across all performance measures. Therefore, the Transformer architecture proves to be a more effective choice for modeling temporal dependencies between transactions within the TREASURE foundation model.

4.4 Negative Sampling Strategy

Efficient negative sampling is crucial for training models on large-scale transaction data, particularly for tasks involving high-cardinality categorical outputs. In this subsection, we compare the effectiveness of different negative sampling strategies. Given that Section 4.2 demonstrated the significant computational burden associated with an independent sampling strategy when employing a large number of negative samples per positive instance, we use a smaller negative sampling size for this strategy in our current comparison. For the shared negative sampling strategy employed in TREASURE, we set the number of negative samples to 1024 per batch. For the independent sampling strategy, we used five negative samples per positive instance. The experiment results are presented in Table 3.

Table 3: Performance comparisons across different negative sampling strategies. The reported metric is Prec@1. Bold values indicate the best results.

	Low Cardinality		High Cardinality	
	Country	Category	City	Merchant
Independent	0.5607	0.4324	0.1370	0.0600
Shared	0.5634	0.4335	0.1892	0.1421

For low-cardinality attributes, the performance is comparable across different sampling methods. This is anticipated, as the choice of negative sampling strategy has a negligible impact on the loss computation for attributes with few distinct values. However, for high-cardinality attributes, we observe that the model trained with independent sampling exhibits significantly poorer performance. This is likely attributable to the insufficient number of negative samples available during the loss computation for these attributes under the independent sampling regime. Consequently, TREASURE is trained using a large pool of negative samples generated on a per-batch basis (shared negative sampling), rather than performing sampling independently for each time step of every instance in a batch.

4.5 Embedding Serving

As TREASURE is a foundation model, a key aspect of its versatility is its ability to provide high-quality embeddings for downstream tasks. In this case study, we evaluate the utility of embeddings generated by TREASURE in a merchant recommendation system. Specifically, the task is to recommend the next merchant a user is likely to interact with. We address this using a standard two-tower recommendation architecture [8, 10, 15, 30], where one tower generates merchant embeddings and the other generates user embeddings.

In these experiments, we compare user and merchant embeddings derived from TREASURE against embeddings learned via supervised training tailored specifically for this recommendation task. The training, validation, and test datasets for this case study were sampled from transactions occurring after September 1, 2022.

This ensures that TREASURE had not encountered these specific user-merchant interactions during its pre-training phase, allowing for a fair evaluation of its generalization capabilities. For clarity, this “pre-training” refers to the general-purpose foundation model training detailed in Section 3.5 and is distinct from the training performed specifically for the recommendation task. Performance is measured using Hit Rate at K (HR@ K) and Normalized Discounted Cumulative Gain at K (NDCG@ K). The experiment results are illustrated in Fig. 7.

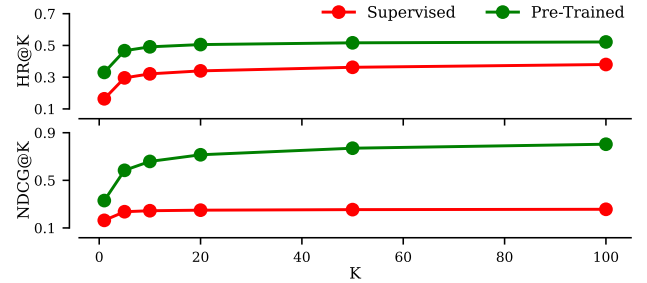


Figure 7: Embeddings generated by TREASURE demonstrate superior effectiveness compared to embeddings from supervised training for recommendation tasks.

The embeddings provided by TREASURE consistently outperform the supervised baseline across various values of K for both HR@ K and NDCG@ K . We computed the performance gain across all settings, observing an average improvement of 104% when using embeddings from TREASURE. This outcome highlights the effectiveness of the pre-training phase, showing that the rich representations learned by TREASURE successfully transfer to specialized downstream applications.

4.6 Embedding Visualization

In addition to objectively verifying the usefulness of the embeddings provided by TREASURE, this subsection qualitatively explores the information they capture. To this end, we visualize individual merchant embeddings by linearly projecting them from their high-dimensional space into a 2D plane. We generated two distinct visualizations to analyze different relational aspects. For the first plot, which focuses on geographic proximity, we created a sample of 500 merchants by randomly selecting 50 from each of 10 popular countries/regions. For the second plot, focusing on business similarity, we created a sample of 500 merchants by selecting 50 from each of 10 popular merchant categories. The resulting scatter plots are presented in Fig. 8; the top plot visualizes the location-based sample, while the bottom visualizes the category-based sample.

Observing the plot for the location-based sample (Fig. 8.top), we see that merchants from the same countries or regions cluster together. These individual clusters also form larger regional super-clusters. More specifically, continental European countries form a large, distinct cluster. The United States and Canada form another, with the United Kingdom located between this North American group and the continental European cluster. Puerto Rico and Mexico form their own cluster, positioned near the one containing the United States and Canada.

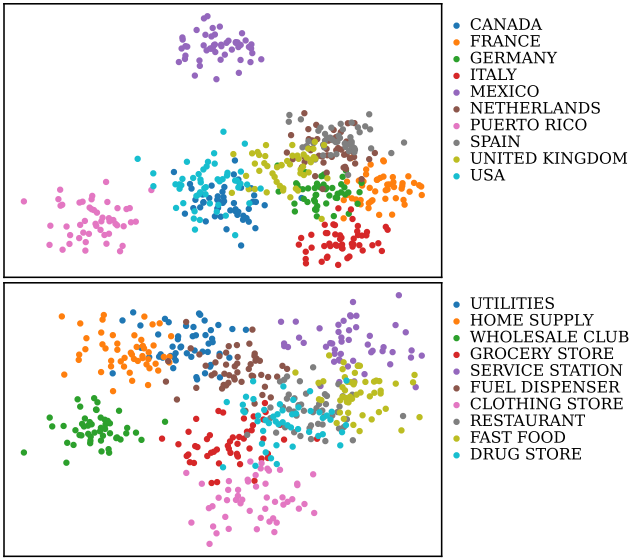


Figure 8: The merchant embedding captures both location and merchant category information.

Likewise, the plot for the category-based sample (Fig. 8.bottom) reveals logical groupings based on business type. For instance, drug stores and grocery stores are in close proximity, an intuitive finding that reflects the modern retail strategy where grocery stores incorporate pharmacies and drug stores expand their food and household staple selections. Similarly, fast-food and full-service restaurants are clustered together, reflecting their shared role as food providers. In contrast, wholesale clubs and clothing stores form distinct clusters that are more distant from the others, likely reflecting their specialized business models; for example, wholesale clubs offer a broad range of goods while clothing stores serve a narrow, specific market. These visual findings suggest that the embeddings learned by TREASURE capture meaningful geographic and categorical relationships between merchants, reflecting real-world market structures and consumer patterns.

In light of multiple existing works that leverage visualization and interaction for improved embedding analysis and interpretation [21, 38], we developed a graphical user interface (GUI) to visualize embeddings in 3D. A screenshot of this tool is presented in Fig. 9. The GUI allows for the interactive projection of embeddings into a 3D space using various algorithms (e.g., PCA [18], t -SNE [25], UMAP [16]) and supports dynamic coloring of points based on their associated metadata. This interface has proven to be an effective tool for rapidly generating and testing hypotheses about the latent structures within the embedding space.

4.7 Data and Model Scaling Analysis

In this section, we analyze the impact of scaling on TREASURE’s performance along two axes: training dataset size and model size. To study data scaling, we trained models on datasets of varying sizes, created by sampling transactions from an increasing number of distinct cardholders. For model scaling, we varied the model’s size by adjusting its hidden dimensions. Each resulting model’s

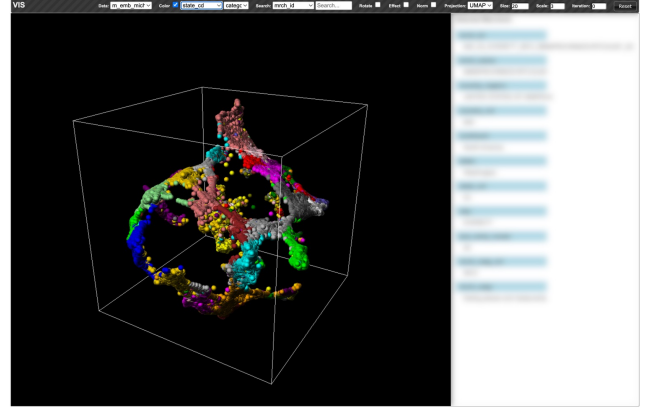


Figure 9: We developed a GUI to explore the embedding space. In the figure, merchant embeddings are visualized and color-coded according to their locations. Please note that the meta-information for each merchant shown in the right panel is blurred to protect privacy.

performance was evaluated using an index that quantifies its relative improvement over the currently deployed system on critical payment-related tasks.

First, we examine the impact of data scaling, with results depicted in Fig. 10. The figure clearly shows a positive correlation between the training dataset size and model performance. This result strongly suggests that further increasing the dataset scale is a promising avenue for developing an even more powerful foundation model for transaction data.

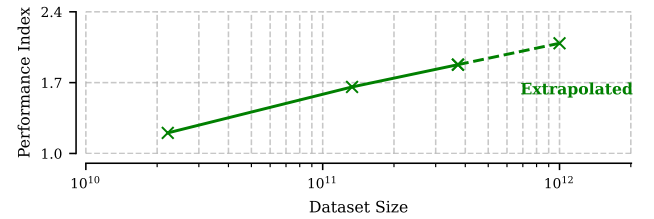


Figure 10: Model performance scales with dataset size, with an extrapolated point for a trillion-sized dataset.

Next, we analyze the effects of model scaling, as shown in Fig. 11. Increasing the model size also benefits the performance of TREASURE. However, unlike the trend observed with data scaling, the performance gains from increasing model size appear to diminish and eventually saturate. This suggests that for a given dataset size, there may be a point of diminishing returns for simply increasing model parameters without a corresponding increase in data.

5 Conclusion

In this paper, we outlined the design of TREASURE, a foundation model specifically engineered for transaction data. TREASURE simultaneously captures both consumer behavior and payment network signals (e.g., response codes, system flags), providing information crucial for applications such as accurate recommendation

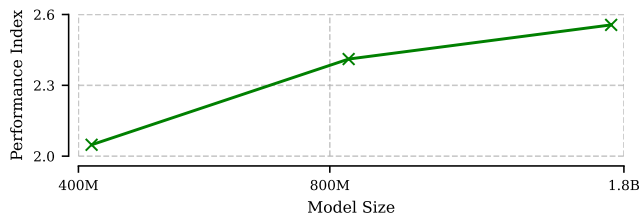


Figure 11: Performance scaling with model size, using 16-bit parameter precision.

systems and abnormal behavior detection. TREASURE has demonstrated promising performance as a foundation model, both when utilized as a standalone system and as an embedding provider. As a standalone abnormal behavior detection system, TREASURE outperformed the currently deployed system by 111%. When leveraged as an embedding provider, its generated embeddings improved recommendation model performance by 104%. The insights derived from developing TREASURE can inform the creation of foundation models for tabular data in other domains. Future work includes several promising directions: enhancing TREASURE’s performance through continued scaling [34] and training strategy refinements; exploring graph-based modeling approaches to leverage multi-hop relationships between entities (e.g., cards interacting with shared merchants [34]; investigating optimization techniques such as quantization and efficient attention mechanisms to reduce inference latency; integrating TREASURE into LLM-based systems to enhance their capabilities in handling transaction data [7, 35]; and adopting in-context learning to combat data drift [36].

References

- [1] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. 2022. Flamingo: a visual language model for few-shot learning. *Advances in neural information processing systems* 35 (2022), 23716–23736.
- [2] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. 2021. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258* (2021).
- [3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [4] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078* (2014).
- [5] Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. 2024. A decoder-only foundation model for time-series forecasting. In *Forty-first International Conference on Machine Learning*.
- [6] DeepSeek-AI. 2025. DeepSeek-V3. <https://huggingface.co/deepseek-ai/DeepSeek-V3> Accessed: 2025-5-9.
- [7] Xiran Fan, Zhimeng Jiang, Chin-Chia Michael Yeh, Yuzhong Chen, Yingdong Dou, Menghai Pan, and Yan Zheng. 2025. Enhancing Foundation Models in Transaction Understanding with LLM-based Sentence Embeddings. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*. 903–911.
- [8] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 639–648.
- [9] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* 9, 8 (1997), 1735–1780.
- [10] Yifan Hu, Yehuda Koren, and Chris Volinsky. 2008. Collaborative filtering for implicit feedback datasets. In *2008 Eighth IEEE international conference on data mining*. IEEE, 263–272.
- [11] Hugging Face. 2025. Llama4. https://huggingface.co/docs/transformers/model_doc/llama4 Accessed: 2025-5-9.
- [12] Kazuki Irie. 2024. Why Are Positional Encodings Nonessential for Deep Autoregressive Transformers? Revisiting a Petroglyph. *arXiv preprint arXiv:2501.00659* (2024).
- [13] Ju-yeong Ji and Ravin Kumar. 2024. Gemma explained: An overview of Gemma model family architectures. <https://developers.googleblog.com/en/gemma-explained-overview-gemma-model-family-architectures/> Accessed: 2025-5-9.
- [14] Amirhossein Kazemnejad, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Payel Das, and Siva Reddy. 2023. The impact of positional encoding on length generalization in transformers. *Advances in Neural Information Processing Systems* 36 (2023), 24892–24928.
- [15] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 42, 8 (2009), 30–37.
- [16] Leland McInnes, John Healy, and James Melville. 2018. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426* (2018).
- [17] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748* (2018).
- [18] Karl Pearson. 1901. LIII. On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin philosophical magazine and journal of science* 2, 11 (1901), 559–572.
- [19] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PMLR, 8748–8763.
- [20] Kashif Rasul, Arjun Ashok, Andrew Robert Williams, Arian Khorasani, George Adamopoulos, Rishika Bhagwatkar, Marin Bilos, Hena Ghonia, Nadhir Hassen, Anderson Schneider, et al. 2023. Lag-llama: Towards foundation models for time series forecasting. In *R0-FoMo: Robustness of Few-shot and Zero-shot Learning in Large Foundation Models*.
- [21] Archit Rathore, Sunipa Dev, Jeff M Phillips, Vivek Srikanth, Yan Zheng, Chia Michael Yeh, Junpeng Wang, Wei Zhang, and Bei Wang. 2024. VERB: Visualizing and interpreting bias mitigation techniques geometrically for word representations. *ACM Transactions on Interactive Intelligent Systems* 14, 1 (2024), 1–34.
- [22] Oleksandr Shchur, Marin Bilos, and Stephan Günnemann. 2019. Intensity-free learning of temporal point processes. *arXiv preprint arXiv:1909.12127* (2019).
- [23] Piotr Skalski, David Sutton, Stuart Burrell, Iker Perez, and Jason Wong. 2023. Towards a foundation purchasing model: Pretrained generative autoregression on transaction sequences. In *Proceedings of the Fourth ACM International Conference on AI in Finance*. 141–149.
- [24] Boris Van Bruegel and Mihaela Van Der Schaar. 2024. Why tabular foundation models should be a research priority. *arXiv preprint arXiv:2405.01147* (2024).
- [25] Laurens Van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of machine learning research* 9, 11 (2008).
- [26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [27] Visa Inc. 2020. Smarter STIP (Stand-in-Processing). <https://usa.visa.com/dam/VCOM/regional/na/us/about-visa/research/documents/smarter-stip.pdf> Accessed: 2025-5-8.
- [28] Visa Inc. 2024. Visa Fact Sheet. <https://corporate.visa.com/content/dam/VCOM/corporate/documents/about-visa-factsheet.pdf> Accessed: 2025-5-5.
- [29] Visa Inc. 2025. Visa Intelligent Commerce. <https://corporate.visa.com/en/products/intelligent-commerce.html> Accessed: 2025-5-8.
- [30] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. 2019. Neural graph collaborative filtering. In *Proceedings of the 42nd international ACM SIGIR conference on Research and development in Information Retrieval*. 165–174.
- [31] Wikipedia contributors. 2025. ISO 3166-1 numeric. Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/ISO_3166-1_numeric Accessed: 2025-5-17.
- [32] Yazheng Yang, Yuqi Wang, Guang Liu, Ledell Wu, and Qi Liu. 2023. Unitab: A universal pretraining protocol for tabular foundation model in data science. *arXiv preprint arXiv:2307.09249* (2023).
- [33] Chin-Chia Michael Yeh, Xin Dai, Huiyuan Chen, Yan Zheng, Yujie Fan, Audrey Der, Vivian Lai, Zhongfang Zhuang, Junpeng Wang, Liang Wang, et al. 2023. Toward a foundation model for time series data. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 4400–4404.
- [34] Chin-Chia Michael Yeh, Mengting Gu, Yan Zheng, Huiyuan Chen, Javid Ebrahimi, Zhongfang Zhuang, Junpeng Wang, Liang Wang, and Wei Zhang. 2022. Embedding compression with hashing for efficient representation learning in large-scale graph. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4391–4401.
- [35] Chin-Chia Michael Yeh, Vivian Lai, Uday Singh Saini, Xiran Fan, Yujie Fan, Junpeng Wang, Xin Dai, and Yan Zheng. 2025. Empowering Time Series Forecasting with LLM-Agents. *arXiv preprint arXiv:2508.04231* (2025).

- [36] Chin-Chia Michael Yeh, Uday Singh Saini, Junpeng Wang, Xin Dai, Xiran Fan, Yujie Sun, Jiarui Fan, and Yan Zheng. 2025. TiCT: A Synthetically Pre-Trained Foundation Model for Time Series Classification. *arXiv preprint arXiv:2511.19694* (2025).
- [37] Dongyu Zhang, Liang Wang, Xin Dai, Shubham Jain, Junpeng Wang, Yujie Fan, Chin-Chia Michael Yeh, Yan Zheng, Zhongfang Zhuang, and Wei Zhang. 2023. Fata-trans: Field and time-aware transformer for sequential tabular data. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 3247–3256.
- [38] Yan Zheng, Junpeng Wang, Chin-Chia Michael Yeh, Yujie Fan, Huiyuan Chen, Liang Wang, and Wei Zhang. 2023. Embeddingtree: Hierarchical exploration of entity features in embedding. In *2023 IEEE 16th Pacific Visualization Symposium (PacificVis)*. IEEE, 217–221.